

Engineering Journal



Ключевые факты:

- **Контроллер:** Хаб LEGO Mindstorms EV3 + 2 мотора
- **Язык программирования:** Python (pybricks)
- **Версии дизайна:** V1(Arduino) => V2 => V3 (текущая, LEGO EV3)
- **Привод:** Задний мотор + LEGO-дифференциал
- **Рулевое управление:** Передний мотор, схема Аккермана
- **Сенсоры:** Датчик цвета, ультразвуковой датчик, PiXu Cam 2.1

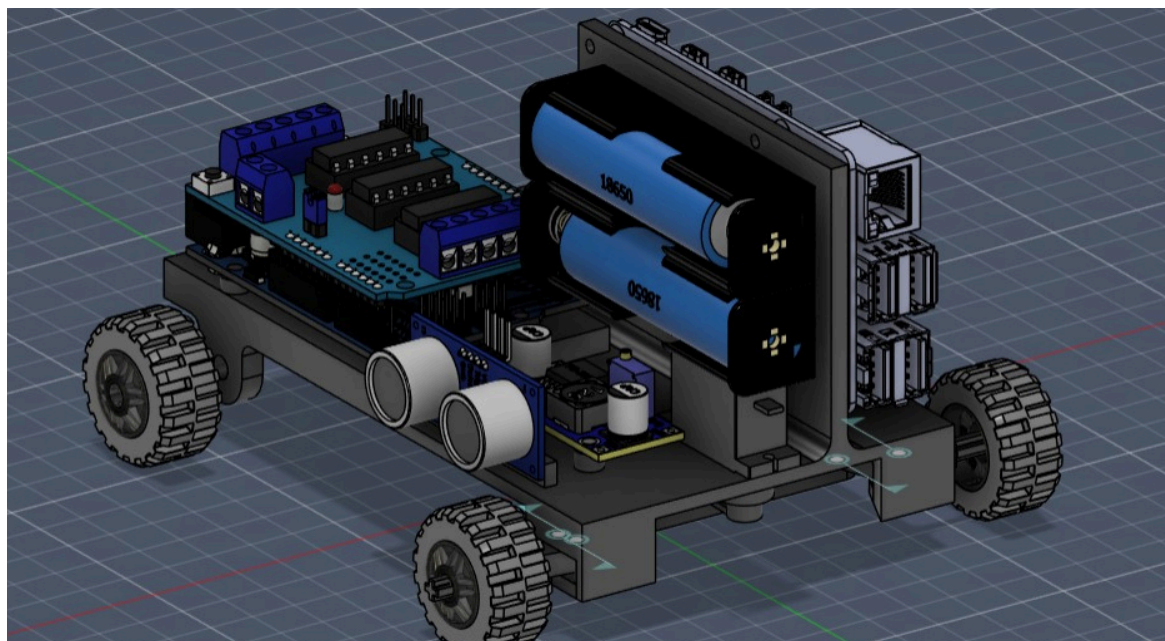
1. Мобильность и механическое проектирование

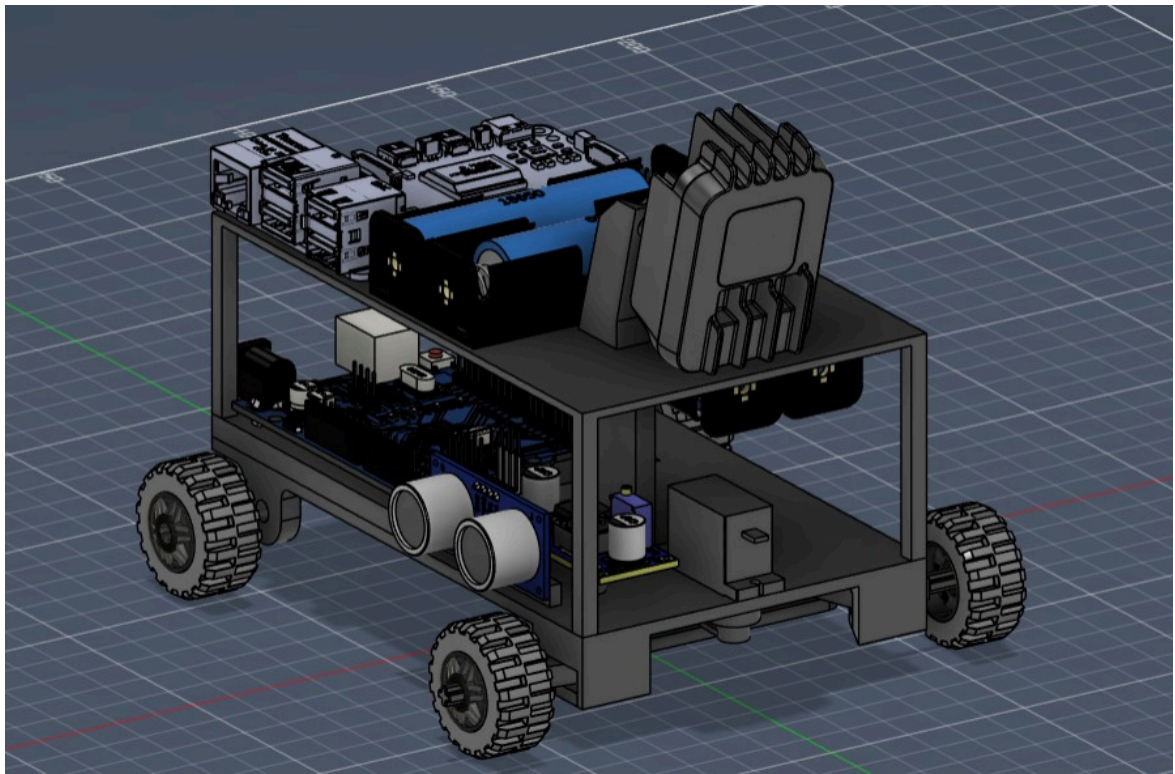
Задний мотор передаёт тягу через LEGO-дифференциал, обеспечивая равное распределение момента в повороте. Передний мотор отдельно управляет рулением по схеме Аккермана, где внутреннее и внешнее колесо поворачиваются на разный угол, что уменьшает проскальзывание. Используются колёса LEGO 86652c01.

Итерация	Описание	Решение / Рекомендация
V1	Прототип на Arduino	Ненадёжный, не соответствовал регламенту
V2	Первая сборка на EV3	Ось переднего колеса слишком свободная, рама расшатывалась. Добавить камеру для выполнения заданий
V3 (Текущая)	Усиление крепления оси балками	Рекомендация на будущее: деталь LEGO Half-Bush #32124 внутри самого дифференциала для устранения причины люфта.

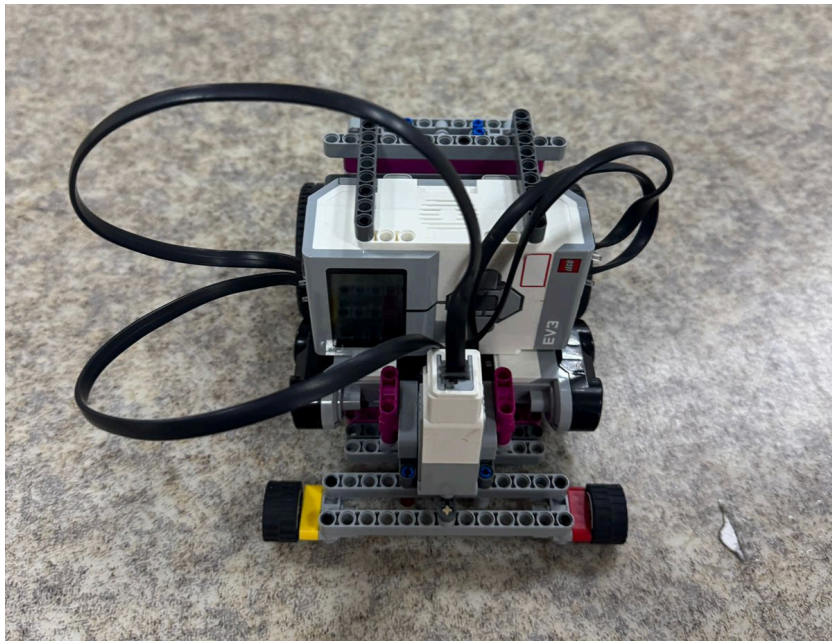
Пробуксовка колёс на повороте решена сменой покрытия колёс и снижением скорости на повороте.

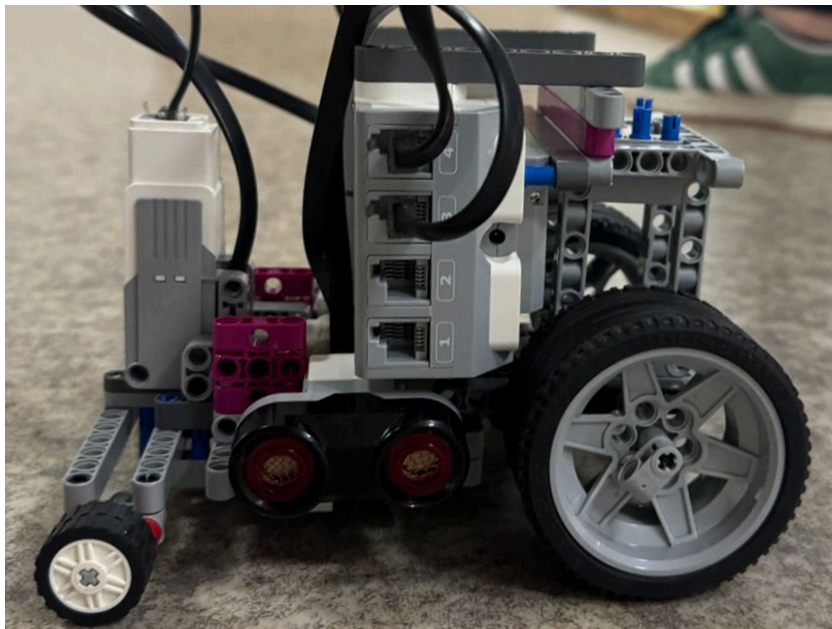
V1:



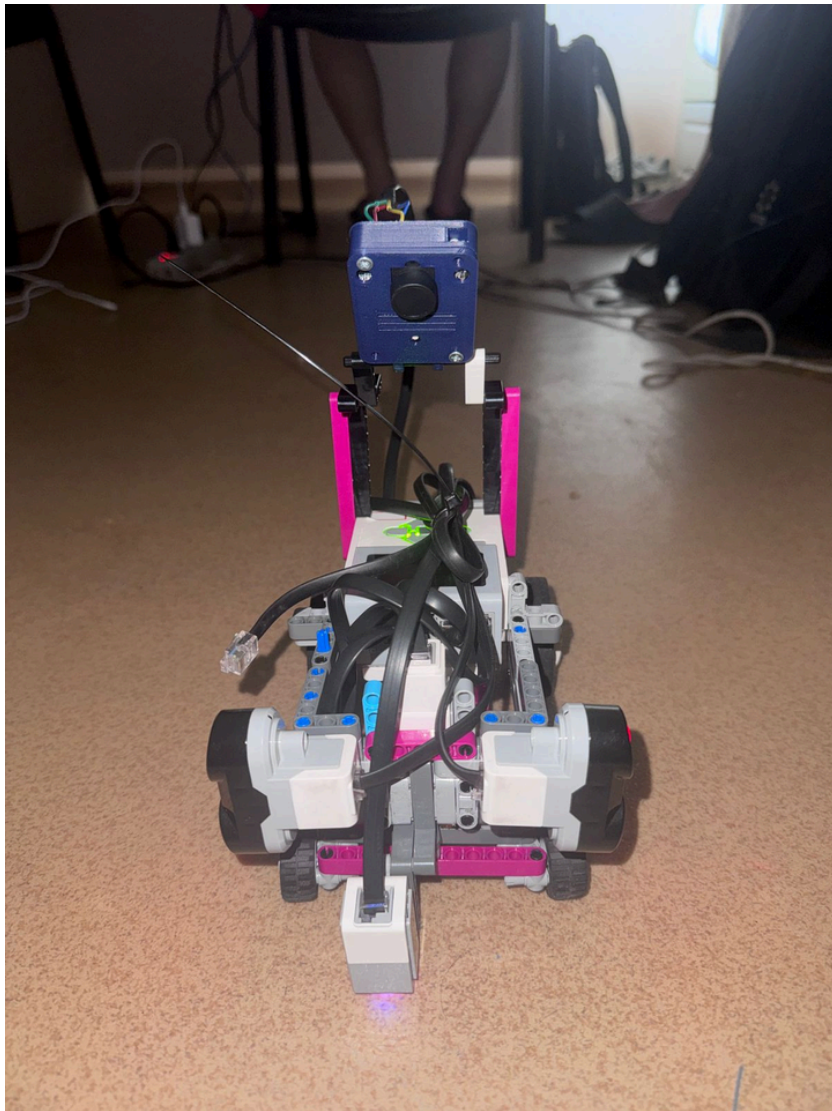


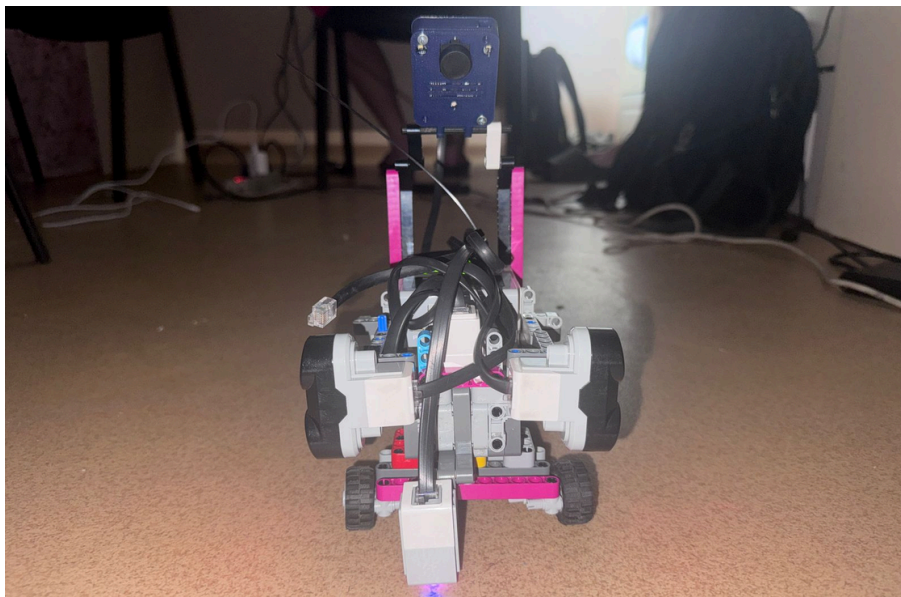
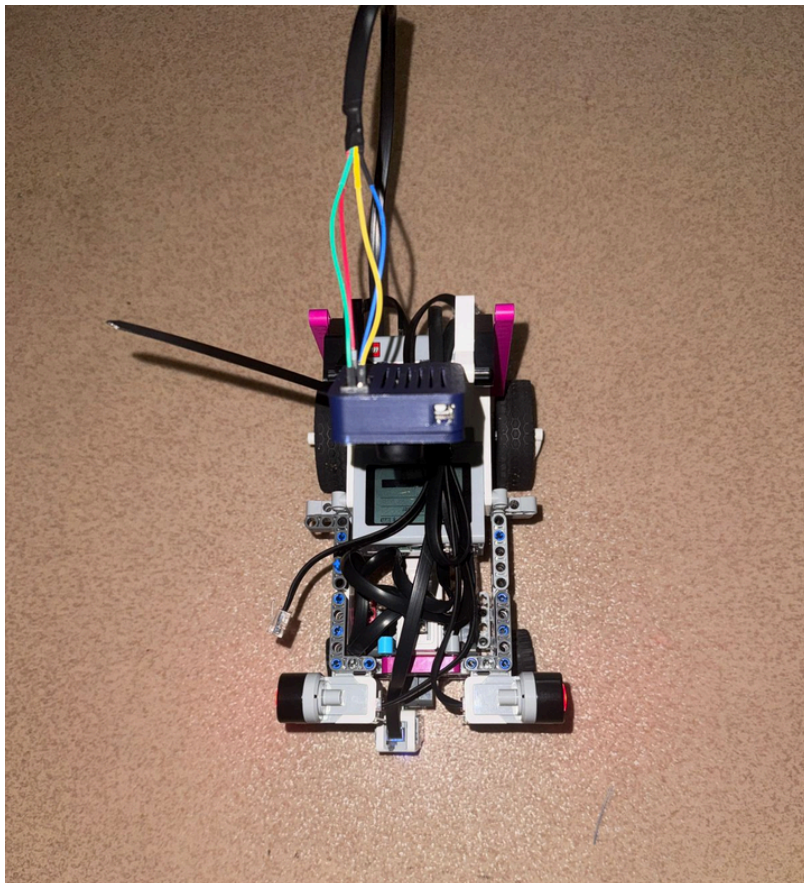
V2:





V3:





Инженерное решение:

Усиление крепления оси переднего колеса балками и использование LEGO Half-Bush #32124 внутри дифференциала для устранения люфта.

2. Архитектура питания и сенсоров

Роли разделены между двумя системами восприятия:

- **Датчик цвета** (спереди, направлен вниз) читает оранжевые/синие метки трассы и определяет сторону поворота.
- **PiXu Cam 2.1** (направлена на внешние объекты, не на трассу) определяет цвет препятствия и сторону объезда.

Замечено: Разряд батареи заметно снижает скорость работы. Мера: проверка заряда хаба перед заездом.

Калибровка:

- Обучать цветовые сигнатуры PiXu Cam под освещение соревнования с разных углов.
- Устанавливать ультразвуковой датчик строго перпендикулярно стенам (отклонение не более 5°) и проверять на известных расстояниях (20/30/40 см).
- Датчик цвета крепить жёстко на высоте 0,5–1 см над трассой.

Инженерное решение:

Разделение функций восприятия между датчиком цвета и PiXu Cam 2.1, с проверкой заряда батареи перед заездом.

3. Архитектура ПО и стратегия преодоления препятствий

Код написан на Python (pybricks). Текущая реализация: последовательный скрипт с if/else. Рекомендуется переход к реализации в виде конечного автомата:

1. Поиск метки
2. Следование по трассе
3. Поворот по метке (bang-bang логика на оранжевый/синий)
4. Обход препятствия (PiXu Cam)
5. Восстановление после потери линии

Открытые вопросы и риски:

- **Порог срабатывания ультразвука:** пока не финализирован.
- **Потеря объекта камерой PiXu Cam:** открытый риск. Рекомендуемое решение: подтверждать объект 2-3 кадрами подряд и делать повторные попытки чтения с таймаутом, а не реагировать на один кадр.

4. Системное мышление и инженерные решения

Карта подсистем включает: мобильность, питание, восприятие, вычисления, конструкция.

Таблица решений:

Критерий	Отклонено	Выбрано
Платформа	Arduino	LEGO EV3
Архитектура восприятия	Без камеры	PiXu Cam 2.1

Ограничения:

- Всего 4 моторных и 4 сенсорных порта EV3.
- Ограниченное количество деталей LEGO и бюджет.
- Ограниченное время до соревнования.

Реестр рисков:

- Разряд батареи
- Расшатывание рамы
- Пробуксовка колёс
- Потеря объекта камерой

Инженерное решение:

Выбор LEGO EV3 как основной платформы и интеграция PiXu Cam 2.1 для расширенных возможностей восприятия, с учётом ограничений по портам и деталям.

5. Воспроизводимость и GitHub

Репозиторий содержит 5+ коммитов, но README требует расширения (цель 5000+ символов). Коммит-сообщения общие ('update', 'fix'). Схемы подключения отсутствуют.

Рекомендуемая структура репозитория:

Таблица BOM (Bill of Materials):

Позиция	Наименование	Количество	Примечание
1	LEGO Mindstorms EV3 Hub	1	
2	LEGO Motors (Large)	2	
3	LEGO Technic Wheels	4	86652c01
4	LEGO Technic Differential	1	
5	Pixy Cam 2.1	1	
6	LEGO Color Sensor	1	
7	LEGO Ultrasonic Sensor	1	

Чек-лист доработок перед сдачей:

- Расширить README.
- Добавить схемы подключения.
- Детализировать коммит-сообщения.
- Финализировать порог срабатывания ультразвука.
- Реализовать механизм устойчивой детекции объектов Pixy Cam.
- Провести финальную калибровку всех сенсоров.
- Заполнить таблицу BOM.

Инженерное решение:

Структурирование репозитория по папкам для улучшения воспроизводимости и добавление полного списка компонентов (BOM) с чек-листом доработок.

6. Теория управления и кинематика

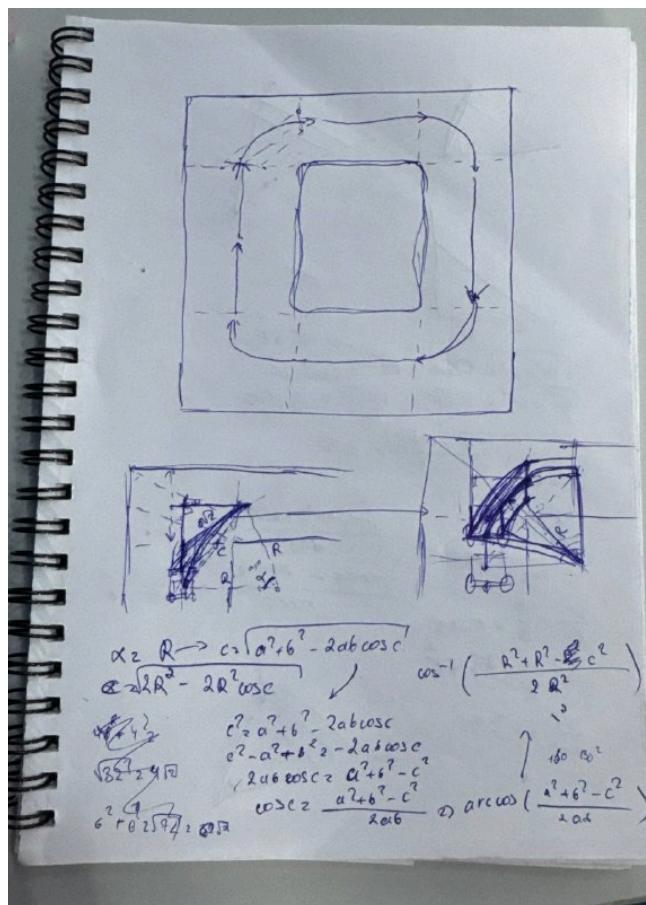
Задний мотор передаёт момент через дифференциал, который сам распределяет угловые скорости колёс по сопротивлению качению: внешнее колесо на повороте раскручивается быстрее внутреннего ($\omega_{\text{outer}} > \omega_{\text{inner}}$) без участия программы — синхронизация скоростей решается механически, а не программным контуром.

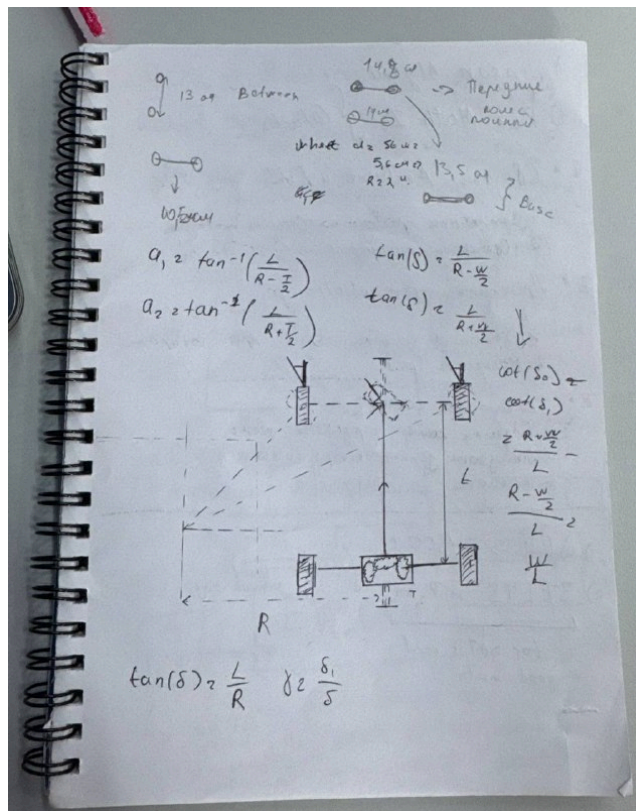
Геометрия Аккермана выражается через котангенсы углов поворота колёс: $\cot(\theta_{\text{outer}}) - \cot(\theta_{\text{inner}}) = d / w$, где d — колея, w — колёсная база. Рулевая рейка реализует это

соотношение механически за счёт трапеции рычагов, поэтому программа управляет не углами левого и правого колеса по отдельности, а положением одного виртуального центрального колеса.

На старте, пока робот не привязан к стене, центрирование идёт сразу по двум дальномерам через нелинейную функцию $f(x) = -2 \cdot \sqrt{(11 \cdot x) + 100}$, где x — расстояние в см. Целевой угол рейки: $\text{target} = 1.3 \cdot f(\text{chap}) - 1.3 \cdot f(\text{rast})$, ограничен $\pm 28^\circ$. Корневая зависимость работает как самонастраивающийся коэффициент усиления: чувствительность руля растёт вблизи стены и падает на расстоянии, что убирает паразитное рысканье на прямых участках без переключения режимов регулятора.

После определения стороны трассы включается классический П-регулятор удержания стены на дистанции 28 см: $e(t) = (\text{distance} - 28) \cdot K_p$, где $K_p = -2$ для левой стены и $+2$ для правой. Так как рейка управляется через энкодер мотора, регулятор берёт рассогласование $\Delta = e(t) - \text{position}(\text{motor})$ и ограничивает его $\text{clamp}(\Delta, -35, 35)$, защищая механику рейки от резких скачков сигнала.





Геометрия Аккермана: поворот вокруг ICC

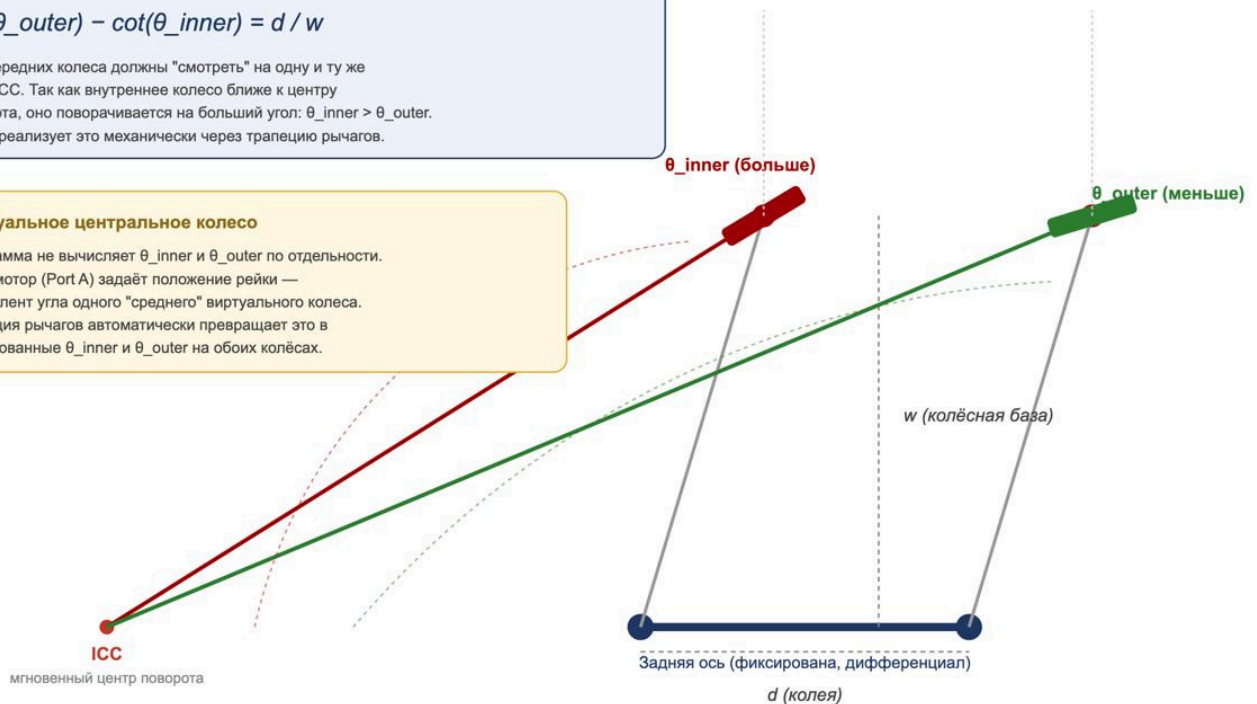
Условие Аккермана (отсутствие юза):

$$\cot(\theta_{\text{outer}}) - \cot(\theta_{\text{inner}}) = d / w$$

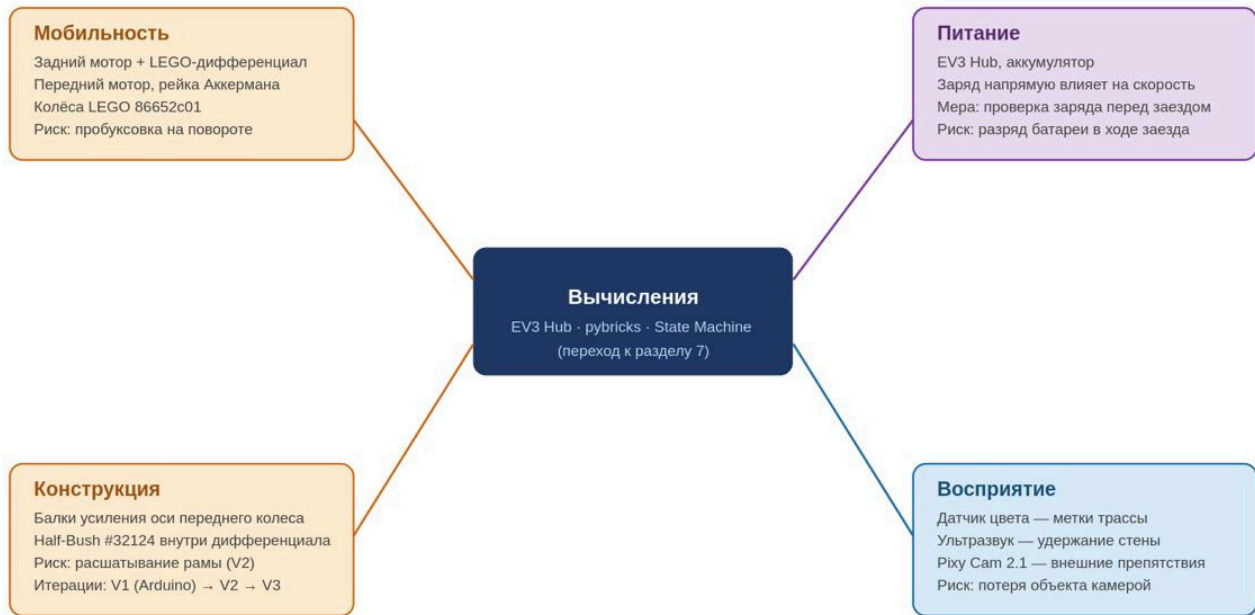
Оба передних колеса должны "смотреть" на одну и ту же точку ICC. Так как внутреннее колесо ближе к центру поворота, оно поворачивается на больший угол: $\theta_{\text{inner}} > \theta_{\text{outer}}$. Рейка реализует это механически через трапецию рычагов.

Виртуальное центральное колесо

Программа не вычисляет θ_{inner} и θ_{outer} по отдельности. Один мотор (Port A) задаёт положение рейки — эквивалент угла одного "среднего" виртуального колеса. Трапеция рычагов автоматически превращает это в согласованные θ_{inner} и θ_{outer} на обоих колёсах.



Карта подсистем робота



Карта портов EV3 Hub



Принцип распределения портов

Функции тяги и рулевого управления физически разнесены по разным моторным портам (A — руль, B — тяга), что исключает конфликт ПИД-контуров в одном моторном контуре. Восприятие также разнесено: датчик цвета отвечает только за разметку трассы (порт 1), Piku Cam — только за внешние препятствия (порт 3). Из 4+4 доступных портов используется 2 моторных и 3 сенсорных — остаётся резерв на 2 порта (C/D, 4) для будущих доработок без изменения текущей архитектуры.

7. Конечный автомат движения и одометрический фильтр

Алгоритм построен как трёхэтапный конечный автомат, где переходы определяются датчиками и одометрией, а не таймерами — это снижает зависимость от трения, заряда батареи и точности стартовой установки.

Stage 1=> старт и сканирование: центрирование по двум дальномерам, пока датчик цвета не зафиксирует первый маркер.

Stage 2 => движение с привязкой к одной стене: направление объезда (по/против часовой) не задаётся в коде заранее, а определяется первым цветным маркером (синий/чёрный → LEFT, оранжевый/красный → RIGHT) — это убирает ошибку человеческого фактора при настройке перед заездом.

Stage 3 => финишный створ: после засчитанной финальной линии датчик цвета отключается, и ещё около 60 итераций цикла удержания стены доводят робота до полной остановки внутри финишного коридора, не обрывая движение точно на пересечении последней линии.

Один и тот же маркер может быть прочитан датчиком цвета несколько раз подряд из-за дребезга на границе цвета. Защита от повторного счёта реализована не задержкой по времени, а по одометрии: энкодер маршевого мотора сбрасывается в нуль перед каждым засчитанным маркером, и новое срабатывание засчитывается только после поворота вала больше чем на 1000° расстояние, заведомо превышающее ширину маркера. Это делает фильтр нечувствительным к скорости движения робота.

Конечный автомат движения (State Machine)

STAGE 1 — Старт и сканирование маркера

- Центрирование по двум датчикам:
 $f(x) = -2\sqrt{11x} + 100$, $target = 1.3 \cdot f(chap) - 1.3 \cdot f(rast)$
- Непрерывный опрос датчика цвета пола
- Ограничение угла рейки: $clamp(target, -28^\circ, 28^\circ)$

Зафиксирован цвет маркера (синий/оранжевый)

STAGE 2 — Привязка к одной стене (навигация по кругу)

- $direction = LEFT \rightarrow$ П-регулятор по chap, цель 28 см
- $direction = RIGHT \rightarrow$ П-регулятор по rast, цель 28 см
 $e(t) = (distance - 28) \cdot Kp$, $\Delta = e(t) - position(motor)$
 $clamp(\Delta, -35^\circ, 35^\circ)$
- Счёт линий с одометрическим кулдауном:
 $motor.position > 1000^\circ$ перед каждым новым счётом
- direction определяется ОДНОКРАТНО в Stage 1 (см. выше)

Счётчик линий достиг значения 11

STAGE 3 — Финишный створ

- Датчик цвета полностью игнорируется
- Удержание стены ещё 60 итераций цикла (по одометрии)
- Полная остановка motor_a и motor_b

Цель: доехать до полного останова внутри финишного коридора, а не оборвать движение точно на пересечении последней линии

Почему переходы по датчикам/одометрии, а не по таймеру

Жёсткие тайминги зависят от трения, заряда батареи и точности стартовой установки робота — любое отклонение от номинальных условий сдвигает момент переключения состояния. Привязка к датчику цвета и энкодеру делает переходы инвариантными к этим факторам: state machine переключается по физическому событию (маркер, пройденный угол), а не по предположению "робот должен быть здесь через N секунд".
Кулдаун $>1000^\circ$ энкодера защищает от дребезга датчика цвета на одном маркере.